EN3160 Assignment 2 on Fitting and Alignment
Ransika L.G.C. 220514C

1. Blob Detection using Laplacian of Gaussian and Scale-Space Extrema

- First, the image is blurred using a Gaussian filter. This reduces noise and smooths details.(The amount of blurring is controlled by the parameter σ)
- Next, we apply the Laplacian operator (which calculates second derivatives) is to the smoothed image. (It detects rapid changes in intensity in a smoothed image)
- Instead of just one scale (σ), the algorithm repeats steps 1 and 2 for different values of σ this builds a scale-space cube.
- Blobs are identified as points that are local maxima (bright blobs) or minima (dark blobs) in the 3D scale-space cube (i.e., across position and scale).
- For each point (x,y,σ), the algorithm checks if it has the highest response compared to its 26 neighbors (8 in 2D, plus neighbors in adjacent scales).

```python
# Define sigma range for scale-space (expected blob radii: sqrt(2)*sigma)
sigma_list = np.linspace(2.0, 10.0, 10)

# Build LoG scale-space
h, w = gray.shape
log_cube = np.zeros((h, w, len(sigma_list)))
for i, sigma in enumerate(sigma_list):
    gauss = gaussian_filter(gray, sigma)
    log_img = laplace(gauss)
    log_cube[:, :, i] = (sigma ** 2) * log_img  # scale-normalized

# Non-maximum suppression in 3D scale-space
maxima = maximum_filter(log_cube, size=(3, 3, 3))
detected = (log_cube == maxima)

# Threshold weak detections
threshold = 0.03
strong = np.abs(log_cube) > threshold
blobs = np.argwhere(detected & strong)

# Sort by response magnitude (to get largest, strongest blobs)
blob_responses = np.abs([log_cube[x,y,z] for x, y, z in blobs])
blobs_sorted = blobs[np.argsort(blob_responses)[::-1]]  # descending
```

2. Line and Circle fitting

(a): Line Fitting with RANSAC

How to Choose this Threshold

The assignment specifies noise with standard deviation $\sigma = 1.0$ for line points. For 95% confidence with Gaussian noise, $t = 1.96\sigma \approx 2.0$. For 99% confidence, $t = 2.58\sigma \approx 2.6$. I used a threshold of $t = 2.0$, which corresponds to capturing approximately 95% of true inliers.

```
def ransac_line(X, threshold=2.0, max_iterations=1000, min_inliers=40):
        distances = compute_line_distances(X, a, b, d)
        inlier_mask = distances < threshold  # <-- ERROR THRESHOLD APPLIED HERE
        n_inliers = np.sum(inlier_mask)
```

How to Choose Minimum Consensus

The dataset has 50 line points and 50 circle points (100 total). The expected inlier ratio is $w = \frac{50}{100} = 0.5$. The minimum consensus should be $d \approx (0.8 \text{ to } 1.0) \times w \times N$. For this dataset, $d \approx 0.8 \times 0.5 \times 100 = 40$. I used min_inliers = 40 as the expected threshold.

```
for i in range(max_iterations):
      # ... model fitting code ...

      #CHECK IF CONSENSUS SET IS LARGE ENOUGH
      if n_inliers > best_inliers:  # <-- IMPLICITLY CHECKING CONSENSUS SIZE
          best_inliers = n_inliers
          best_params = (a, b, d)
          best_mask = inlier_mask
          best_sample = sample_points.copy()
```

(b): Circle Fitting with RANSAC

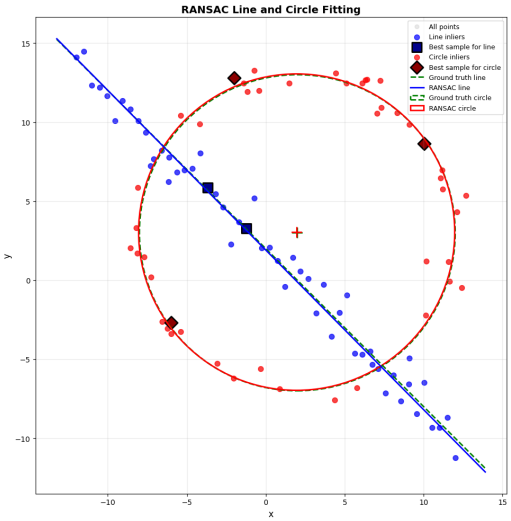
RANSAC Circle Parameters

Parameter	Location in Code	Value	Calculation / Description
Radial Error Threshold	`threshold` in `ransac_circle()`	2.0	Based on circle noise $s = r/16 = 0.625$, using $t = 2.58\sigma \approx 1.61$ for 99% confidence.
Minimum Consensus	`min_inliers` in `ransac_circle()`	40	$0.8 \times 50 = 40$ expected circle points.
Number of Iterations	`max_iterations`	1000	$N = \frac{\log(1-0.99)}{\log(1-(1-e)^3)}$, where $e \approx 0$.

(d) If the Circle is Fitted First Instead of the Line

1. **Higher outlier ratio:** Circle RANSAC faces $\sim 50\%$ outliers, requiring more iterations for robust estimation.
2. **Computational cost:** More iterations are needed for 99% confidence. The number of iterations $N = \frac{\log(1-p)}{\log(1-(1-e)^m)}$, where $p = 0.99$, $e = 0.5$, $m = 3$, giving $N \approx 35$ vs ~ 7 for line-first.
3. **Contaminated second fit:** Removing circle inliers leaves both line points and circle outliers, complicating line fitting.
4. **Sequential dependency:** The simpler line model (2 points) is easier to fit first, followed by the more complex circle (3 points); line fitting converges faster.
5. **Better separation:** Line-first removes a well-defined subset, leaving cleaner points for circle fitting.
6. **Conclusion:** Line-first is more efficient and provides better separation of the two geometric models.

(c): Visulization



3. Planar Homography and Image Superimposition

This example demonstrates real-world advertising applications where virtual product placement on billboards is increasingly used in broadcast media, streaming services, and virtual environments. Billboard advertising relies on strategic placement in high-traffic areas for maximum exposure, and digital homography techniques enable dynamic content updates without physical installation

Homography Matrix Analysis

Affine Part (Upper-Left 2×2):
Rotation: Slight ($\approx -0.7°$), nearly fronto-parallel and
Scaling/Shear: $\sim$27% vertical compression with mild shear.

Translation:
Shifts the poster 186 px right and 178 px down.

Perspective Terms:
Minor perspective distortion—billboard slightly angled to camera.

Overall, the matrix shows realistic geometric alignment with minimal rotation and mild perspective effects, suitable for accurate virtual ad placement.

$$H_{\text{computed}} = \begin{bmatrix} 0.992 & -0.012 & 186.0 \\ 0.250 & 0.728 & 178.0 \\ 3.301 \times 10^{-4} & -1.969 \times 10^{-5} & 1 \end{bmatrix}$$

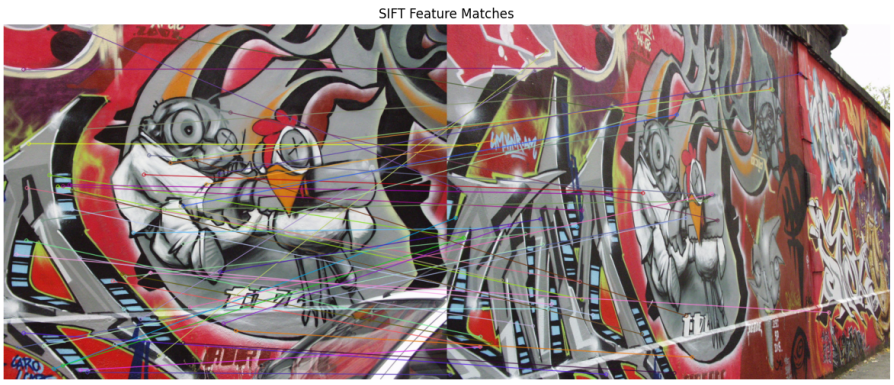
$$\text{Affine: } \begin{bmatrix} 0.992 & -0.012 \\ 0.250 & 0.728 \end{bmatrix}$$

$$\text{Translation: } \begin{bmatrix} 186.0 \\ 178.0 \end{bmatrix}$$

$$\text{Perspective: } \begin{bmatrix} 3.3 \times 10^{-4} & -2.0 \times 10^{-5} \end{bmatrix}$$

4. SIFT Matching and Stitching

(a) Compute and Match SIFT Features Between the Two Images



(b) Homography Comparison

RANSAC Homography:

$$H_{\text{RANSAC}} = \begin{bmatrix} -1.2951 & -0.1953 & 315.0244 \\ -3.9273 & 0.8222 & 390.2896 \\ -0.0105 & 0.0019 & 1 \end{bmatrix}$$

Ground Truth Homography:

$$H_{\text{GT}} = \begin{bmatrix} 0.6254 & 0.0578 & 222.0122 \\ 0.2224 & 1.1652 & -25.6056 \\ 0.000492 & -0.0000365 & 1 \end{bmatrix}$$

Homography Difference:

$$\Delta H = \begin{bmatrix} 1.9205 & 0.2530 & 93.0123 \\ 4.1497 & 0.3430 & 415.8952 \\ 0.0110 & 0.0019 & 0 \end{bmatrix}$$

Performance Summary

- **Inlier Ratio:** 7/87 (8.0%)
- **Mean Error:** 57.29 (high deviation)
- Homography shows large rotation and translation errors.

Key Insights

- Low inlier ratio indicates poor RANSAC consensus.
- Extreme viewpoint change and clustered matches caused instability.
- 2,000 iterations were insufficient—about 17,000 needed for 8% inliers.
- 3-pixel threshold too strict; adaptive threshold may help.

(c) Stiched images